Car Detection Methods and Their Evaluation Metrics

Ali Najafzadeh

April 2025

Abstract

The detection of cars through video is a basic computer vision task which is helpful in monitoring traffic and developing self-driving vehicles. There are some traditional approaches relying on intensity differences to recognize moving objects from a static background. Although these methods have a great time complexity, they are not suitable for dynamic lighting conditions or movements (Fradi & Dugelay, 2012). Gaussian Mixture Models (GMM) are better at dynamic lightings because it models pixels as a distribution over time (Fradi & Dugelay, 2012). Modern deep learning detectors such as You Only Look Once (YOLO) enables real-time object detection with convolutional neural networks. YOLOv4 achieves better speed and accuracy when using the CSP-Darknet53 (Wang et al., 2021). The Tiny-YOLOv4 detector is a lightweight alternative for fast real-time performance yet it requires some trade-offs for detection accuracy (Medina et al., 2024). This report presents four different car detection techniques coded in MATLAB inlcuding threshold-based background subtraction, GMM background subtraction, Tiny-YOLOv4, and YOLOv4 with CSP-Darknet53.

Keywords: Car Detection, Background Subtraction, Gaussian Mixture Model (GMM), YOLOv4, Tiny-YOLOv4, Deep Learning, Computer Vision, Object Detection, Intersection-over-Union (IoU), Precision, Recall, F1-Score

1 Methods

1.1 Thresholding with Background Subtraction

Background subtraction and Thresholding is a basic technique which enables car detection. The reference background image needs to be stored before the method subtracts each new frame from this reference. After the pixel-wise difference is calculated, thresholding is applied to identify objects in motion. Fradi and Dugelay, 2012 outline this straightforward method as one of the simplest background subtraction techniques is to calculate an average image of the scene with no moving objects, subtract each new frame from this image, and threshold the result (Fradi & Dugelay, 2012). The simple nature of thresholding makes it straightforward to implement and fast yet it fails under conditions with noise and shadows along with changes in illumination (Fradi & Dugelay, 2012).

1.2 Gaussian Mixture Model (GMM)

The Gaussian Mixture Model (GMM) was used by Fradi and Dugelay, 2012 as an enhanced background subtraction algorithm which manages dynamic backgrounds (Fradi & Dugelay, 2012). Through GMM background modeling pixels demonstrate their intensities through a distribution of Gaussian values instead of a fixed value. Pixels can have different values depending on waving tree branches, cars temporarily stopped or moving shadows. The system automatically updates Gaussians by adjusting means and variances and dynamically shifting weights throughout the process and will be adaptive across each pixel. The image pixels that cannot match the Gaussian elements will be identified as foreground objects representing moving cars (Fradi & Dugelay, 2012). The GMM approach is the primary method for outdoor video surveillance applications when dealing with moving backgrounds (Fradi & Dugelay, 2012). Fradi and Dugelay, 2012 developed an advanced version of the original GMM by providing automatic Gaussians adaptation per pixel which boosted computation performance and precision levels (Fradi & Dugelay, 2012).

1.3 You Only Look Once Object Detection

Object detections like YOLO (You Only Look Once) are widely used for car detection. YOLO is a one-stage CNN-based detector that directly predicts

bounding boxes and class probabilities in a single evaluation of the network (Bochkovskiy et al., 2020). The YOLO analysis by Bochkovskiy et al., 2020 showed that a CNN could detect objects faster by framing detection as a regression problem instead of a region classification task (Bochkovskiy et al., 2020). Since then, the YOLO family has evolved through several versions. YOLOv4 introduced numerous improvements in accuracy (Bochkovskiy et al., 2020). YOLOv4 uses the CSP-Darknet53, which incorporates Cross-Stage Partial (CSP) connections to improve learning efficiency and reduce computation (Wang et al., 2021). Tiny-YOLOv4 is a quick performance variant although it causes some reduction in accuracy and it has been engineered as a reduced layer version of YOLOv4 (Medina et al., 2024). It will raise its operational speed to over 6 times compared to full YOLOv4 (Jiang et al., 2023). These CNN detectors are amongest the leading detection algorithms for vehicle detection by integrating deep learning.

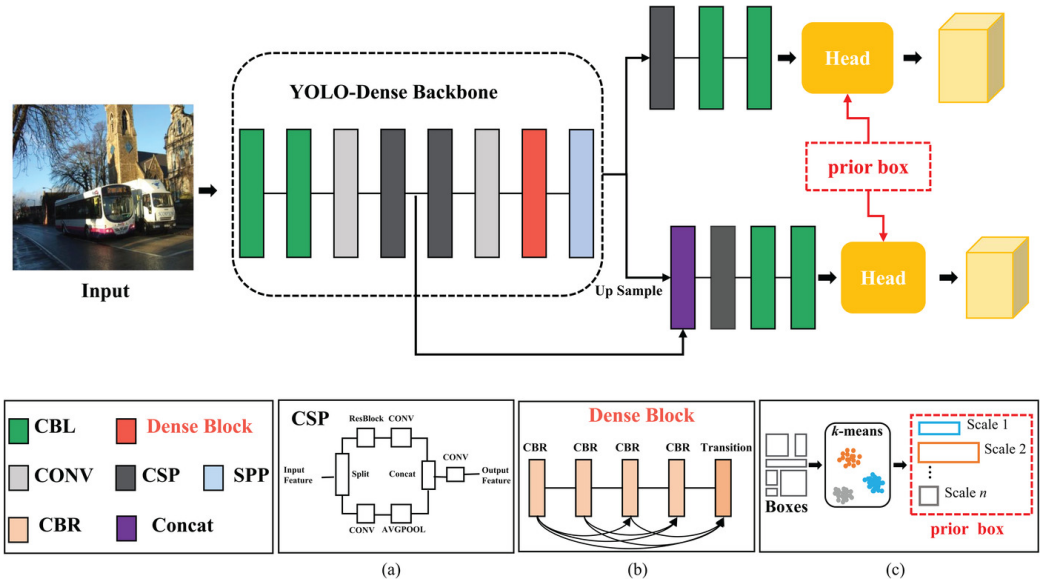

Figure 1: The overview of the YOLOv4-dense network which contains two parts, namely, YOLO-dense Backbone and YOLOv4-dense head (Jiang et al., 2023).

1.4 Evaluation Metrics

Standard metrics are used to quantify performance:

- True Positives (TP): correctly detected car overlapping a ground-truth car with IoU
- False Positives (FP): detection reported where there is no car.
- False Negatives (FN): fails to detect cars present in the frame.
- Precision: the ability of a model to identify only relevant objects

$$\text{Precision} = \frac{\text{TP}}{(\text{TP} + \text{FP})}$$

- Recall: the proportion of actual targets that were detected

$$\text{Recall} = \frac{\text{TP}}{(\text{TP} + \text{FN})}$$

  A perfect object detection system should aim to maintain both strong precision and recall levels but in practice there is often a trade-off.

- F1-Score: The F1 metric gives a single value reflecting overall performance balance.

$$\text{F1} = \frac{2 \cdot (\text{Precision} \cdot \text{Recall})}{(\text{Precision} + \text{Recall})}$$

- Intersection over Union (IoU): IoU measures the overlap between the predicted bounding box and the ground-truth bounding box of a car. If this overlap exceeds a chosen threshold the detection is counted as a correct detection (TP). Otherwise it may be counted as a false positive.

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

  A commonly used threshold in evaluations is IoU = 0.5. Higher IoU thresholds like 0.75 has a more stringent localization and boxes must overlap more to count as TP. Benchmarks like MS COCO report Average Precision averaged over multiple IoU thresholds from 0.5 to 0.95 (Padilla et al., 2021).

2 Results

Each detection method was benchmarked with 120-frames of video with ground-truth bounding boxes for cars. The evaluation was conducted using a fixed IoU threshold of 0.5. The performance metrics for each method are presented in Table 1. The YOLOv4 with CSP-Darknet53 architecture achieved the highest overall performance 0.9660 as precision, 0.8465 as recall, and F1-score of 0.9023, and it was the highest among all methods. This shows its deep learning ability to generalize across varying scenes. The Tiny-YOLOv4 model illustrated perfect precision 1.0000. However, its recall was significantly lower 0.5056, indicating many cars were missed. The GMM detector had a balanced performance with F1-score of 0.7579 which outperforms thresholding in both precision and overall robustness. Occasional multi-box detections and false positives due to background dynamics were also observed for it. The threshold-based detector performed acceptably with F1-score of 0.7377 but its sensitivity to background noise and lighting variation reduced recall. The parameters were optimized for this video and may not generalize well to different scenes and it may not be applicable under real-world conditions.

Method	**Precision**	**Recall**	**F1-Score**
Thresholding	0.7681	0.7097	0.7377
GMM	0.8361	0.6931	0.7579
YOLO tiny-yolov4-coco	1.00	0.5056	0.6716
YOLO csp-darknet53-coco	0.9660	0.8465	0.9023

Table 1: Performance metrics for detection methods. The evaluation was performed using a fixed IoU threshold of 0.5.

3 Figures

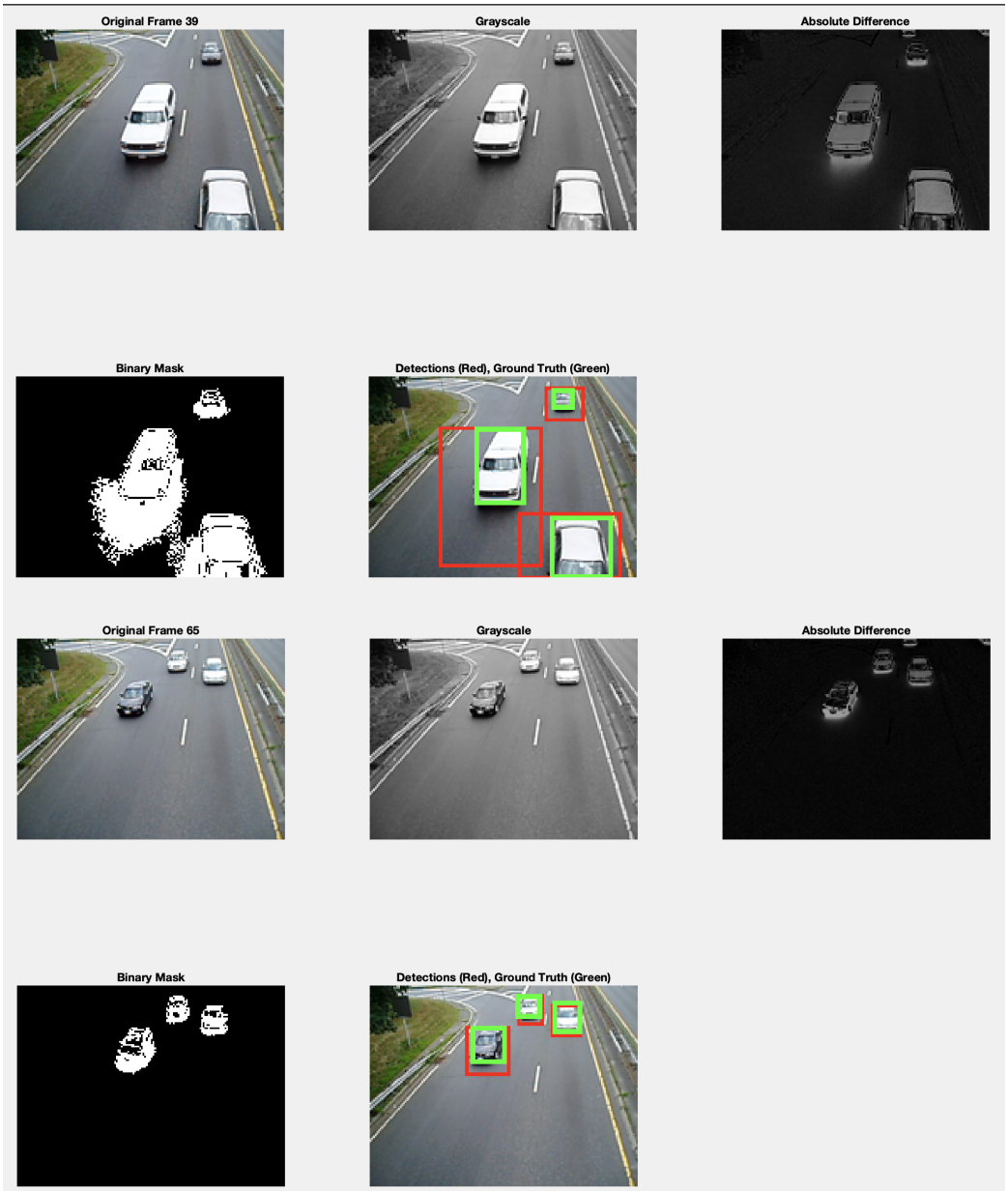

Figure 2: Car detection using threshold-based background subtraction. These figures illustrate the detection of moving cars by subtracting a static background from the current frame then applying a pixel-wise threshold. White regions represent the pixels that are above the threshold. Noise and shadows are also visible which shows the method's sensitivity to illumination changes.

Figure 3: Gaussian Mixture Model (GMM) background subtraction. Here, each pixel is modeled as a mixture of Gaussians to account for dynamic backgrounds. The GMM approach adapts to environmental variations better than thresholding and reduces false positives. Bounding boxes enclose the detected vehicles. GMM can track moving cars in moderately complex scenes. Multiple detections for a single vehicle may occur because of the background changes.

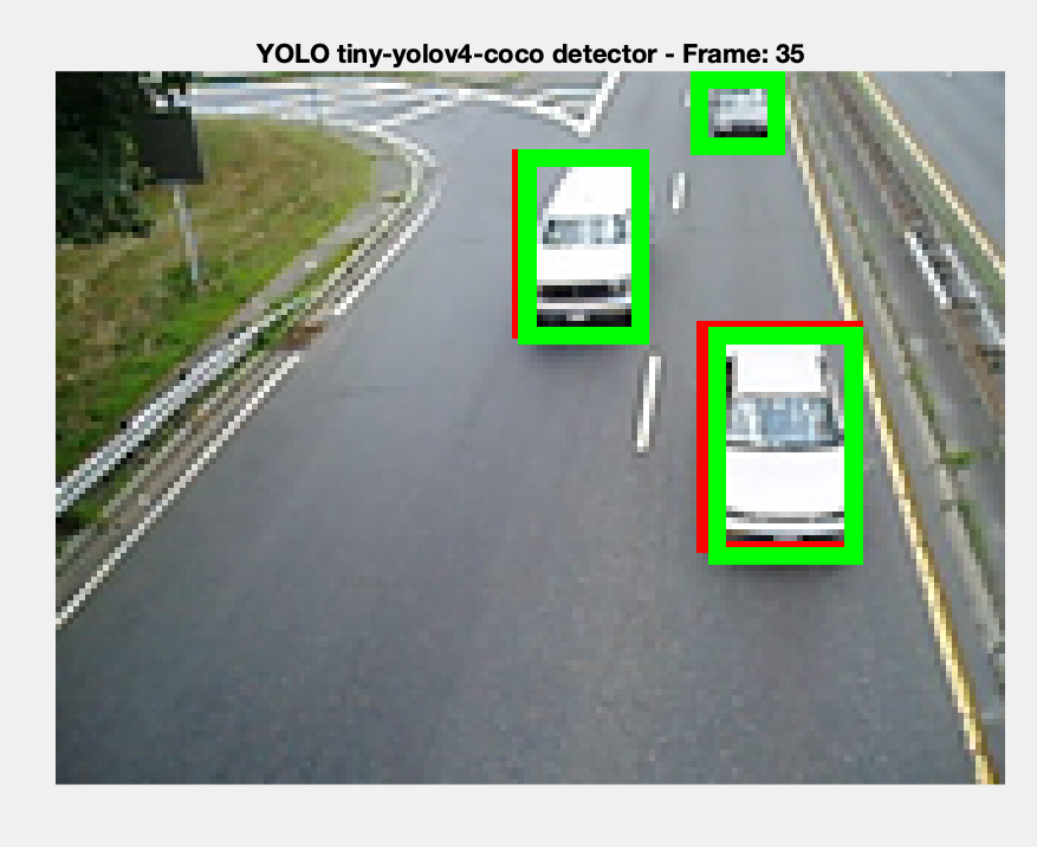

Figure 4: Car detection using Tiny-YOLOv4 model. Tiny-YOLOv4 object detection algorithm is a lightweight variant of YOLOv4 optimized for speed and real-time performance. However, due to its simplified architecture and reduced number of convolutional layers it often has a lower recall especially in lower-resolutions.

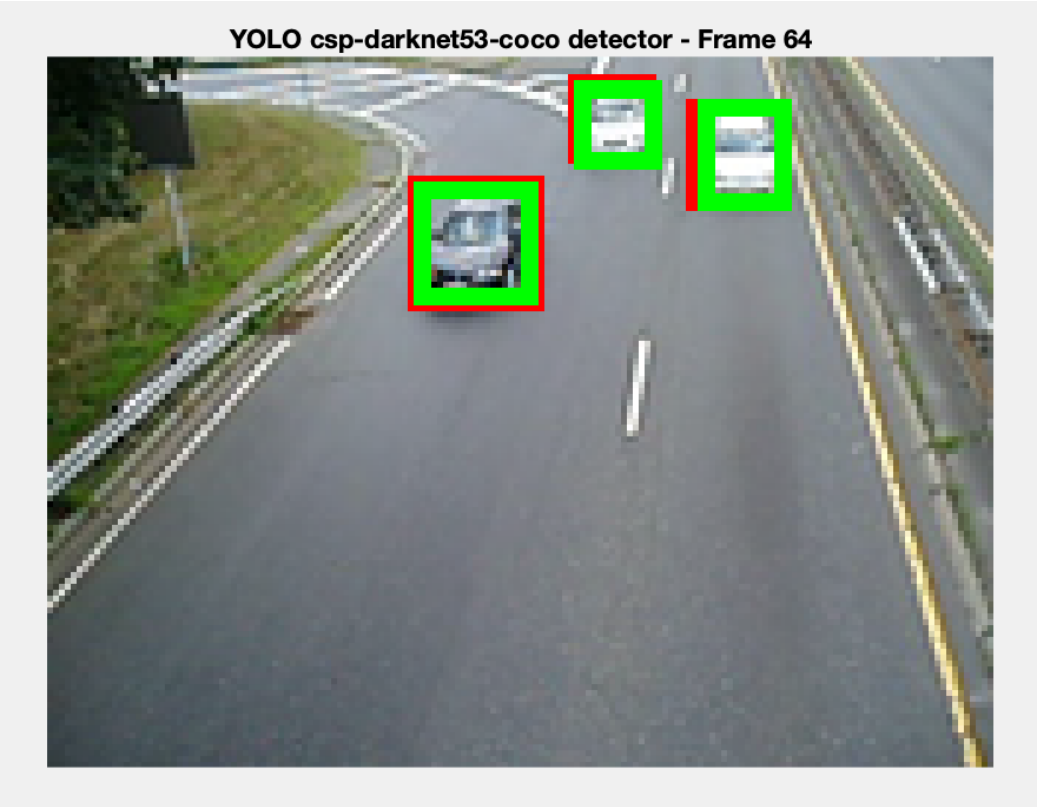

Figure 5: Car detection using YOLOv4 with CSP-Darknet53. Compared to its lightweight variant, YOLOv4 combines high precision and recall and the best overall performance and matches the ground truth but it requires a high computing power.

References

Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). Yolov4: Optimal speed and accuracy of object detection. https://arxiv.org/abs/2004.10934

Fradi, H., & Dugelay, J.-L. (2012). Robust foreground segmentation using improved gaussian mixture model and optical flow. *2012 International Conference on Informatics, Electronics Vision (ICIEV)*, 248–253. https://doi.org/10.1109/ICIEV.2012.6317376

Jiang, Y., Li, W., Zhang, J., Li, F., & Wu, Z. (2023). Yolov4-dense: A smaller and faster yolov4 for real-time edge-device based object detection in traffic scene. *IET Image Processing*, *17*(2), 570–580. https://doi.org/https://doi.org/10.1049/ipr2.12656

Medina, A., Bradley, R., Xu, W., Ponce, P., Anthony, B., & Molina, A. (2024). Learning manufacturing computer vision systems using tiny yolov4. *Frontiers in Robotics and AI*, *11*. https://doi.org/10.3389/frobt.2024.1331249

Padilla, R., Passos, W. L., Dias, T. L. B., Netto, S. L., & da Silva, E. A. B. (2021). A comparative analysis of object detection metrics with a companion open-source toolkit. *Electronics*, *10*(3). https://doi.org/10.3390/electronics10030279

Wang, C.-Y., Bochkovskiy, A., & Liao, H.-Y. M. (2021). Scaled-yolov4: Scaling cross stage partial network. https://arxiv.org/abs/2011.08036

Appendix

A Code

Here is the MATLAB code used for the car detection algorithm:

```
% Just setting up the figure size manually
figure('Position', [100, 100, 1200, 700]);

% Load in ground truth data and the video
load('export_ground_truth.mat');
videoName = 'traffic.mj2';
videoReader = VideoReader(videoName);

% usable frames
```

```
gtFrameCount = height(gTruth.LabelData);
videoFrameCount = videoReader.NumFrames;
usableFrames = min(gtFrameCount, videoFrameCount);

% ground truth bounding boxes
gtAllBoxes = cell(usableFrames, 1);

% grab the GT boxes
for frameIdx = 1:usableFrames
    rawBox = gTruth.LabelData.Car{frameIdx};
    if isa(rawBox, "struct")
        rawBox = rawBox.Position;
    end
    if isempty(rawBox)
        rawBox = zeros(0,4);
    end
    gtAllBoxes{frameIdx} = rawBox;
end

% the detectors
fprintf('Running Threshold detector\n');
thresholdBoxes = threshold_detector(videoName, usableFrames,
    gtAllBoxes);

fprintf('Running GMM detector\n');
gmmBoxes = gmm_detector(videoName, usableFrames, gtAllBoxes);

fprintf('Running tiny YOLO\n');
yoloBoxes = yolo_detector(videoName, usableFrames, gtAllBoxes
    );

fprintf('Running csp YOLO\n');
yolov2CarBoxes = yolo_csp_detector(videoName, usableFrames,
    gtAllBoxes);

% IOU threshold for metric calculation
iouThreshold = 0.5;

% Evaluating detectors
thresholdMetrics = evaluate_metrics(thresholdBoxes,
    gtAllBoxes, iouThreshold);
gmmMetrics = evaluate_metrics(gmmBoxes, gtAllBoxes,
    iouThreshold);
yoloMetrics = evaluate_metrics(yoloBoxes, gtAllBoxes,
    iouThreshold);
```

```
yolov2CarMetrics = evaluate_metrics(yolov2CarBoxes, gtAllBoxes, iouThreshold);

% Show results
display_evaluations(thresholdMetrics,gmmMetrics, yoloMetrics, yolov2CarMetrics);

% Thresholding
function allBoxes = threshold_detector(videoFile, totalFrames, groundTruth)
    % Using the first frame as static background
    reader = VideoReader(videoFile);
    refFrame = read(reader, 1);
    % converting it to gray scale
    refGray = rgb2gray(refFrame);
    allBoxes = cell(totalFrames, 1);

    for idx = 1:totalFrames
        currentFrame = read(reader, idx);
        grayFrame = rgb2gray(currentFrame);

        % Difference image (simple background subtraction)
        diffImg = abs(double(grayFrame) - double(refGray));

        % Binary threshold
        binaryMask = imbinarize(uint8(diffImg), 0.1);
        % Remove small items
        binaryMask = bwareaopen(binaryMask, 50);

        % Get bounding boxes
        regions = regionprops(binaryMask, 'BoundingBox');
        boxes = reshape([regions.BoundingBox], 4, []).';

        allBoxes{idx} = boxes;

        % Visualization
        subplot(2,3,1);
        imshow(currentFrame);
        title(sprintf('Original Frame %d', idx));
        subplot(2,3,2);
        imshow(grayFrame);
        title('Grayscale');
        subplot(2,3,3);
        imshow(uint8(diffImg));
        title('Absolute Difference');
```

```
        subplot(2,3,4);
        imshow(binaryMask);
        title('Binary Mask');

        overlayFrame = currentFrame;
        if ~isempty(boxes)
            overlayFrame = insertShape(overlayFrame, 'Rectangle', boxes, 'Color', 'red', 'LineWidth', 2);
        end
        if ~isempty(groundTruth{idx})
            overlayFrame = insertShape(overlayFrame, 'Rectangle', groundTruth{idx}, 'Color', 'green', 'LineWidth', 3);
        end

        subplot(2,3,5);
        imshow(overlayFrame);
        title('Detections (Red), Ground Truth (Green)');
        pause(0.1);
    end
end

% GMM Detector
function allBoxes = gmm_detector(videoPath, frameCap, gt)
    reader = VideoReader(videoPath);
    fgDetector = vision.ForegroundDetector('NumGaussians', 3, 'NumTrainingFrames', 5);
    allBoxes = cell(frameCap, 1);

    for frameNum = 1:frameCap
        current = read(reader, frameNum);
        gray = rgb2gray(current);
        mask = fgDetector.step(gray);
        % Remove small items
        mask = bwareaopen(mask, 60);

        stats = regionprops(mask, 'BoundingBox');
        boxes = reshape([stats.BoundingBox], 4, []).';
        allBoxes{frameNum} = boxes;
        % Visualize
        subplot(1, 2, 1);
        imshow(mask);
        title(sprintf('GMM Foreground Mask - Frame %d', frameNum))
```

```
        % adding mask to show the detected cars
        if ~isempty(boxes)
            current = insertShape(current, 'Rectangle', boxes, 'Color', 'red', 'LineWidth', 2);
        end
        if ~isempty(gt{frameNum})
            current = insertShape(current, 'Rectangle', gt{frameNum}, 'Color', 'green', 'LineWidth', 3);
        end
        subplot(1,2,2); imshow(current);
        title(sprintf('GMM Detector Result - Frame %d', frameNum));
        pause(0.1);
    end
end

% tiny YOLO
function allBoxes = yolo_detector(videoPath, maxFrames, gt)

    reader = VideoReader(videoPath);
    detector = yolov4ObjectDetector("tiny-yolov4-coco");

    allBoxes = cell(maxFrames, 1);
    confidenceThreshold = 0.3;

    for frameIdx = 1:maxFrames
        frame = read(reader, frameIdx);
        [bboxes, scores, labels] = detect(detector, frame);

        % Filter by class and confidence
        carMask = strcmp(cellstr(labels), 'car') & scores > confidenceThreshold;
        carBoxes = bboxes(carMask, :);

        allBoxes{frameIdx} = carBoxes;

        if ~isempty(carBoxes)
            frame = insertShape(frame, 'Rectangle', carBoxes, 'Color', 'red', 'LineWidth', 2);
        end
        if ~isempty(gt{frameIdx})
            frame = insertShape(frame, 'Rectangle', gt{frameIdx}, 'Color', 'green', 'LineWidth', 3);
        end
        subplot(1, 2, 2); imshow(frame);
```

```
        title(sprintf('YOLO tiny-yolov4-coco detector - Frame : %d', frameIdx));
        pause(0.05);
    end
end

% csp YOLO
function allBoxes = yolo_csp_detector(vidFile, totalFrames, gtLabels)

    reader = VideoReader(vidFile);
    detector = yolov4ObjectDetector("csp-darknet53-coco");

    allBoxes = cell(totalFrames, 1);
    minConfidence = 0.3;

    for i = 1:totalFrames
        frame = read(reader, i);
        [bboxes, scores, labels] = detect(detector, frame);

        carBoxes = bboxes(strcmp(cellstr(labels), 'car') & scores > minConfidence, :);

        allBoxes{i} = carBoxes;

        if ~isempty(carBoxes)
            frame = insertShape(frame, 'Rectangle', carBoxes, 'Color', 'red', 'LineWidth', 2);
        end
        if ~isempty(gtLabels{i})
            frame = insertShape(frame, 'Rectangle', gtLabels{i}, 'Color', 'green', 'LineWidth', 3);
        end

        subplot(1, 2, 2); imshow(frame);
        title(sprintf('YOLO csp-darknet53-coco detector - Frame %d', i));
        pause(0.01);
    end
end

% Metrics Evaluation
function metrics = evaluate_metrics(predBoxes, gtBoxes, iouThreshold)
    numFrames = length(gtBoxes);
```

```
    precisions = zeros(numFrames,1);
    recalls = zeros(numFrames,1);
    for k = 1:numFrames
        gt = gtBoxes{k};
        pred = predBoxes{k};
        [p, r] = bboxPrecisionRecall(pred, gt, iouThreshold);
        precisions(k) = p;
        recalls(k) = r;
    end
    precision = mean(precisions);
    recall = mean(recalls);
    f1 = 2 * (precision * recall) / (precision + recall + eps);
    metrics = struct('Precision', precision, 'Recall', recall, 'F1Score', f1);
end


function display_evaluations(thresholdMetrics,gmmMetrics, yoloMetrics, yolov2CarMetrics)
    disp('Thresholding:');
    disp(thresholdMetrics);
    disp('GMM:');
    disp(gmmMetrics);
    disp('YOLO tiny-yolov4-coco detector:');
    disp(yoloMetrics);
    disp('YOLO csp-darknet53-coco detector:');
    disp(yolov2CarMetrics);

    metricsNames = {'Threshold', 'GMM', 'YOLO tiny-yolov4-coco', 'YOLO csp-darknet53-coco'};
    precisions = [thresholdMetrics.Precision, gmmMetrics.Precision, yoloMetrics.Precision, yolov2CarMetrics.Precision];
    recalls = [thresholdMetrics.Recall, gmmMetrics.Recall, yoloMetrics.Recall, yolov2CarMetrics.Recall];
    f1Scores = [thresholdMetrics.F1Score, gmmMetrics.F1Score, yoloMetrics.F1Score, yolov2CarMetrics.F1Score];
    figure('Position', [100, 100, 1200, 700]);
    barData = [precisions; recalls; f1Scores]';
    barHandle = bar(barData, 'grouped');
    set(barHandle, 'FaceColor', 'flat');

    barHandle(1).FaceColor = [0.2 0.6 1];
    barHandle(2).FaceColor = [1 0.6 0];
```

```
    barHandle(3).FaceColor = [0.8 0 0];

    set(gca, 'XTickLabel', metricsNames);
    legend({'Precision', 'Recall', 'F1 Score'}, 'Location', 'Best');
    xlabel('Detector');
    ylabel('Metric Value');
    title('Evaluation Metrics for Different Detectors');
    grid on;
end
```